



Code Review Using Phabricator

Phabricator is a suite of Web based software development collaboration tools. It includes features like bug tracking, repository browsing, change monitoring and a wiki. It is free and open source, and was originally developed by Evan Priestly as an internal tool for Facebook.

I cannot do better than repeat the beginning of an article I wrote five years ago about ReviewBoard: *“When I look back, it isn’t that we were not aware of some of the recommended processes nor did we believe that they were not useful. It is just that the development infrastructure was so poor that we could not effectively implement them. It really boiled down to too much effort for too little returns.”*

One of my regrets is that while teaching a computer lab class, my efforts to integrate ReviewBoard into the process was a miserable failure. In retrospect, it is understandable that in an educational system which is exam and time constrained rather than outcome oriented, this was the likely result.

Learning the code review process at the college level is important for a number of reasons, even though the students will quickly adapt to it depending on the processes followed in the organisations they eventually join. A couple of reasons are:

- Code review is a great way to learn by reading the code of other people, both good and bad!
- It is essential to ensure a distance between one’s ego and the code one writes. Each one needs to get used to and be comfortable at receiving and welcoming criticism and advice, for the code one has written.

Searching the Internet brings us a number of open source solutions for code review. ReviewBoard continues to be a major player. Gerrit, released by Google, appears to be a very good option as well. For me, Phabricator stood out because of its odd name. These three tools use different platforms as well. ReviewBoard uses Python based Django, and Gerrit is a J2EE application, in contrast to the earlier version which also used Python. Finally, Phabricator is built on the LAMP stack! It is based on the internal tools used at Facebook.

The process

All these tools are meant for many developers in a networked environment. So, even for testing on a single machine, you will need to set up a process that will work for many users.

You will need to set up an authoritative repository. As Git is the widely used repository these days, you should stick with it.

As a developer, you will do the following:

- Clone the repository
- Develop the additional code
- Submit it for review
- Push the change to the authoritative repository after the code has been approved post review

A reviewer will need to review your code, offer advice and when convinced by your approach, approve the code.

Installation

Installation of Phabricator is straightforward but a little complex. You can find a guide at <https://secure.phabricator.com/>. Once the installation is done, you will get a notification if any issues were found, on the main page of your Phabricator website. You can fix each one; but during limited testing, choose to ignore the issues related to performance.

Phabricator comprises a set of tools, aside from the Web server, for code review.

As a developer, you will need to use Arcanist. It is included in the above installation. You can follow the *Quickstart* page to set up Arcanist for your project.

You may install Arcanist in any directory. For convenience, create the following link:

```
$ sudo ln -s <install path>/arcanist/bin/arc /usr/local/bin/arc
```

Code changes

First create a Git repository, `~/repo/project`, and then follow the steps shown below:

```
$ mkdir ~/repo/slave
$ cd ~/repo/slave
$ git clone ~/repo/project .
```

Configure Arcanist for the project, if needed, by using the following command:

```
$ arc feature dev
```